

TP3 - Observer (Patrones de Comportamiento)

Materia: Metodología de Sistemas II

Unidad 2: Patrones de Diseño

Alumno: Nicolás Viruel

Lenguaje: Python (sin librerías externas)

1. Problema identificado

InventoryManager conocia por nombre a EmailAlertService, AnalyticsDashboard y AutoReplenishment.

Cada vez que el stock bajaba, llamaba directamente a los tres servicios.

Agregar un canal nuevo (push, SMS) obligaba a editar el manager. Acoplamiento alto y logica duplicada.

2. Diagrama UML - ANTES

InventoryManager

- Atributos concretos: emailService, analytics, replenishment
- Métodos: update_stock, sell_product
- Llama directamente a EmailAlertService, AnalyticsDashboard, AutoReplenishment

Problema: el manager conoce servicios concretos por nombre. Agregar push/SMS obliga a editar InventoryManager.

3. Diagrama UML - DESPUÉS

Interfaz StockObserver (suscriptor)

- Método: on_low_stock(product_id, quantity)

Observadores concretos (implementan StockObserver)

- EmailAlertObserver, AnalyticsObserver, ReplenishObserver, PushNotificationObserver, BrokenObserver

InventoryManager (notificador / publisher)

- Atributo: observers - lista genérica de StockObserver
- Métodos: subscribe, unsubscribe, _notify, update_stock, sell_product
- _notify recorre la lista; try/except dentro del loop por cada observer

Relación: InventoryManager no conoce observadores concretos. Solo publica el evento y cada suscriptor reacciona.

4. Código ANTES (problemático)

```
class InventoryManager:
    def __init__(self):
        self.emailService = EmailAlertService()
        self.analytics = AnalyticsDashboard()
        self.replenishment = AutoReplenishment()

    def update_stock(self, productId, quantity):
        self.stock[productId] = quantity
        if quantity < 10:
            self.emailService.send_low_stock_alert(productId, quantity)
            self.analytics.record_low_stock_event(productId)
```

```

        self.replenishment.trigger_order(productId, 100)

def sell_product(self, productId, sold):
    self.stock[productId] -= sold
    if self.stock[productId] < 10:
        self.emailService.send_low_stock_alert(productId, ...)
        self.analytics.record_low_stock_event(productId)
        self.replenishment.trigger_order(productId, 100)

```

5. Código DESPUÉS (refactorizado completo)

"""

TP3 - Observer | Metodología de Sistemas II

Alumno: Nicolás Viruel

InventoryManager usa lista generica de suscriptores StockObserver.

El try/except esta DENTRO del loop: un observer roto no detiene a los demas.

"""

```

from abc import ABC, abstractmethod

```

```

class StockObserver(ABC):
    @abstractmethod
    def on_low_stock(self, product_id, quantity): ...

```

```

class EmailAlertObserver(StockObserver):
    def on_low_stock(self, product_id, quantity):
        print(f"[EMAIL] Stock bajo: {product_id} (qty={quantity})")

```

```

class AnalyticsObserver(StockObserver):
    def on_low_stock(self, product_id, quantity):
        print(f"[ANALYTICS] Evento registrado: {product_id}")

```

```

class ReplenishObserver(StockObserver):
    def on_low_stock(self, product_id, quantity):
        print(f"[REPLENISH] Orden automatica x100 para {product_id}")

```

```

class PushNotificationObserver(StockObserver):
    """Observador nuevo agregado sin tocar InventoryManager."""

    def on_low_stock(self, product_id, quantity):
        print(f"[PUSH] Alerta movil: {product_id} quedo en {quantity}")

```

```

class BrokenObserver(StockObserver):
    def on_low_stock(self, product_id, quantity):
        raise RuntimeError("error de red simulado")

```

```

class InventoryManager:
    def __init__(self):
        self._stock = {}

```

```

self._observers = []          # lista generica, sin nombres concretos
self._errors = []

def subscribe(self, observer):
    self._observers.append(observer)

def unsubscribe(self, observer):
    self._observers.remove(observer)

def _notify(self, product_id, quantity):
    for observer in self._observers:
        try:                                # DENTRO del loop
            observer.on_low_stock(product_id, quantity)
        except Exception as exc:
            msg = f"Observer {observer.__class__.__name__} fallo: {exc}"
            self._errors.append(msg)
            print(f"[ERROR registrado] {msg}")
            # CONTINUA con el siguiente observer

def update_stock(self, product_id, quantity):
    self._stock[product_id] = quantity
    if quantity < 10:
        self._notify(product_id, quantity)

def sell_product(self, product_id, sold):
    self._stock[product_id] = self._stock.get(product_id, 0) - sold
    if self._stock[product_id] < 10:
        self._notify(product_id, self._stock[product_id])

```

InventoryManager con lista generica: `_observers: list[StockObserver]` sin referencias a clases concretas.

try/except dentro del loop: cada observer es independiente; uno puede fallar sin cortar la cadena.

6. Prueba obligatoria - Observer roto

```

manager = InventoryManager()
manager.subscribe(EmailAlertObserver())
manager.subscribe(AnalyticsObserver())
manager.subscribe(BrokenObserver())          # lanza excepcion
manager.subscribe(ReplenishObserver())
manager.subscribe(PushNotificationObserver())

```

```
manager.update_stock("PROD-001", 5)
```

Resultado esperado y obtenido:

```

=== update_stock('PROD-001', 5) ===
[EMAIL] Stock bajo: PROD-001 (qty=5)
[ANALYTICS] Evento registrado: PROD-001
[ERROR registrado] Observer BrokenObserver fallo: error de red simulado
[REPLENISH] Orden automatica x100 para PROD-001
[PUSH] Alerta movil: PROD-001 quedo en 5

```

```
Errores aislados: ['Observer BrokenObserver fallo: error de red simulado']
```

- EMAIL: OK (antes del roto)
- ANALYTICS: OK (antes del roto)
- BrokenObserver: falla y se registra el error

- REPLENISH: OK (despues del roto, cadena NO se detuvo)
- PUSH: OK (observador extra demuestra extensibilidad)

7. Justificación del patrón

Observer desacopla el sujeto (inventario) de los observadores (email, analytics, reposición). El manager solo publica el evento "stock bajo" recorriendo una lista genérica; no sabe quiénes son los suscriptores. Permite agregar PushNotificationObserver sin modificar InventoryManager. El manejo de errores por observer evita que un servicio caído bloquee el resto.